

Cloud Networking

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How the same networking fundamentals – subnets, routing, addressing – get reimplemented as software in AWS, Azure, and GCP, and what actually changes when "the network" is an API call instead of a cable.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 – 3	What changes in the cloud, VPCs/VNets, and subnets/route tables
4 – 5	Security groups vs NACLs, and connecting VPCs together
6 – 7	Cloud load balancing/DNS, and hybrid connectivity back to on-prem
8 – 9	A worked example building a basic VPC, and reference

How to use this guide: read start to finish for the full picture, or jump to section 4 if you just need security groups vs NACLs clarified. Every diagram is numbered and referenced directly in the surrounding text.

Table of Contents

1. What Changes When the Network Is Software	3
2. VPCs & VNets: The Cloud's Private Network	4
3. Subnets, Route Tables & Internet Gateways	5
4. Security Groups vs NACLs	6
5. VPC Peering & Transit Gateways	7
6. Load Balancing & DNS in the Cloud	8
7. Hybrid Connectivity: VPN & Direct Connect	9
8. Worked Example: Building a Basic VPC	10
9. Quick Reference & Glossary	11

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

What Changes When the Network Is Software

1.1 SAME CONCEPTS, DIFFERENT IMPLEMENTATION

Every fundamental idea from traditional networking – subnets, routing, firewalling, DNS, load balancing – still exists in the cloud. What changes is *how they're created and managed*: instead of racking a physical router or configuring a VLAN on a physical switch, a cloud network is defined entirely through API calls, configuration files, or a web console. There's no cable to trace, no physical port to plug into – the entire topology is **software-defined**, running on infrastructure the cloud provider owns and abstracts away.

1.2 WHAT ACTUALLY CHANGES IN PRACTICE

- **Provisioning speed** – a new subnet or firewall rule takes effect in seconds via an API call, not a maintenance window and a site visit.
- **No physical topology to reason about** – there's no real trunk port or cable run to troubleshoot; issues live in configuration and provider-side abstractions instead.
- **Everything is code-definable** – cloud networks are commonly defined in version-controlled configuration (Terraform, CloudFormation), making the entire network reviewable, repeatable, and auditable like software.
- **Shared underlying infrastructure** – your VPC (Section 2) runs on physical hardware shared with countless other customers, isolated entirely through software boundaries rather than physical separation.

The abstractions are opinionated, and provider-specific. AWS calls it a VPC, Azure calls the same underlying concept a VNet, and each provider's exact feature set, terminology, and default behaviors differ in real, sometimes surprising ways. The mental model transfers across providers; the exact commands and defaults don't.

VPCs & VNets: The Cloud's Private Network

2.1 YOUR OWN ISOLATED SLICE OF THE CLOUD

A **VPC (Virtual Private Cloud, AWS/GCP terminology)** or **VNet (Virtual Network, Azure terminology)** is a logically isolated, private network within a cloud provider's infrastructure – your own private IP address space, entirely separate from every other customer's, even though it physically runs on the same shared hardware. It's conceptually similar to the private RFC 1918 address space from the NAT Deep Dive guide, but provisioned and managed entirely as cloud infrastructure.

2.2 WHAT YOU DEFINE AT THE VPC LEVEL

COMPONENT	ROLE
CIDR block	The overall private IP address range for the VPC (e.g. <code>10.0.0.0/16</code>)
Subnets	Smaller carved-out ranges within the VPC, often one per availability zone (Section 3)
Route tables	Rules controlling where traffic from each subnet is allowed to go (Section 3)
Internet/NAT gateways	Controlled points of entry/exit to the public internet (Section 3)
Security groups / NACLs	Firewall-like traffic filtering at the instance and subnet level (Section 4)

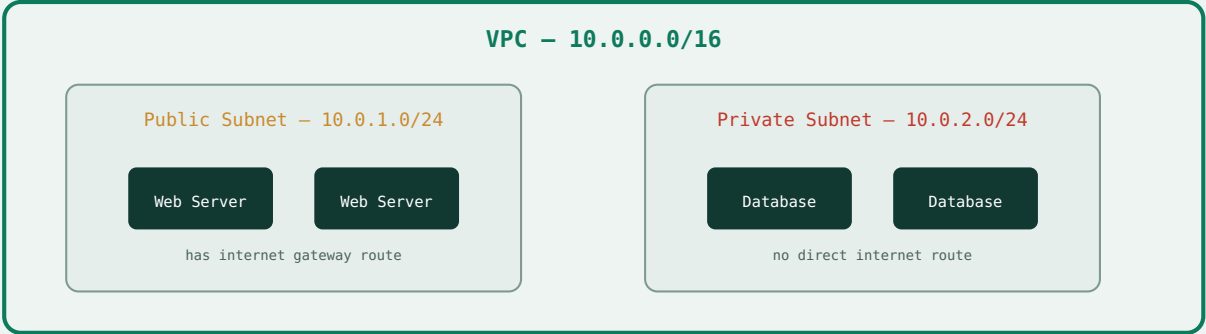


Fig 2.1 – A VPC carved into a public subnet (internet-reachable) and a private subnet (isolated), the most common baseline pattern.

Subnets, Route Tables & Internet Gateways

3.1 PUBLIC VS PRIVATE SUBNETS

A subnet's "public" or "private" status isn't an inherent property – it's purely determined by its **route table**: a **public subnet** has a route sending internet-bound traffic (`0.0.0.0/0`) to an **Internet Gateway**; a **private subnet** simply has no such route, so its instances cannot be reached from, or directly reach, the public internet at all.

3.2 INTERNET GATEWAY VS NAT GATEWAY

An **Internet Gateway** provides two-way internet connectivity – instances in a public subnet can both receive inbound connections and initiate outbound ones. A **NAT Gateway**, placed in a public subnet but referenced by a *private* subnet's route table, provides **outbound-only** internet access for private instances (e.g., so a private database server can still download software updates) – directly analogous to the home-router PAT/NAT covered in the NAT Deep Dive guide, just implemented as managed cloud infrastructure rather than a physical router.

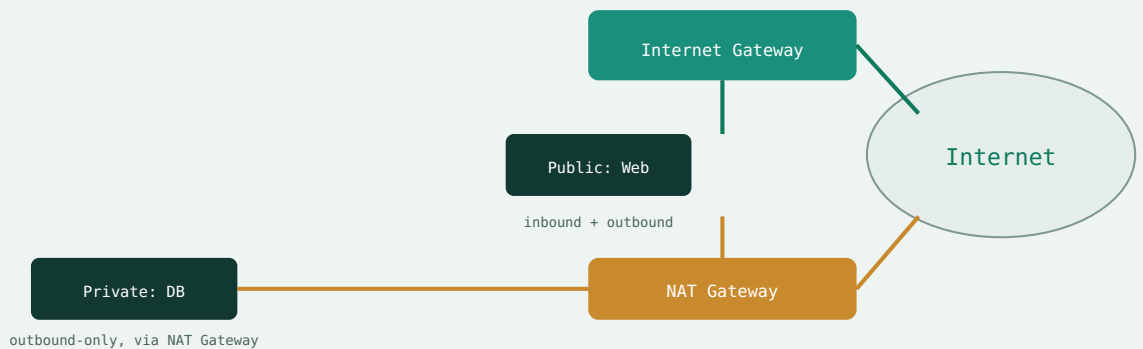


Fig 3.1 – A public subnet routes through an Internet Gateway (both directions); a private subnet routes outbound-only through a NAT Gateway.

3.3 ROUTE TABLES: THE ACTUAL DECISION-MAKER

Every subnet is associated with exactly one route table, and that table's rules are the entire, definitive answer to "where does this subnet's traffic go." There's no separate concept of a "gateway" that's implicitly used – everything is explicit, declarative routing rules, making it possible to audit a subnet's entire reachability just by reading its route table.

Security Groups vs NACLs

4.1 SECURITY GROUPS: STATEFUL, INSTANCE-LEVEL

A **Security Group** is a stateful firewall attached directly to individual instances (or network interfaces). **Stateful** means that if inbound traffic is allowed in, the corresponding outbound reply is automatically allowed back out without needing its own explicit rule – the security group tracks the connection's state, much like the NAT translation table from the NAT Deep Dive guide tracks its own connections.

4.2 NACLs: STATELESS, SUBNET-LEVEL

A **Network ACL (NACL)** operates at the subnet level, filtering all traffic entering or leaving that subnet, and is **stateless** – inbound and outbound rules are evaluated completely independently, so allowing inbound traffic on a port does *not* automatically allow the corresponding outbound reply; that needs its own explicit rule too.

ASPECT	SECURITY GROUP	NACL
Applies to	Individual instances/interfaces	Entire subnet
State tracking	Stateful – replies auto-allowed	Stateless – both directions need explicit rules
Rule evaluation	All matching "allow" rules apply	Ordered rules, first match wins (allow or deny)
Can explicitly deny?	No – allow-only, implicit deny otherwise	Yes – explicit allow and deny rules

Defense in depth: security groups and NACLs are typically used together, not as alternatives – NACLs provide a coarse, subnet-wide safety net, while security groups provide fine-grained, per-instance control. This mirrors the same layered-defense principle from traditional network security (see the main Computer Networks guide's security chapter).

VPC Peering & Transit Gateways

5.1 VPC PEERING: DIRECT, ONE-TO-ONE

VPC Peering creates a direct network connection between exactly two VPCs, letting resources in each communicate using private IP addresses as if they were on the same network – without any traffic crossing the public internet. Peering connections are **not transitive**: if VPC A peers with VPC B, and VPC B peers with VPC C, A cannot reach C through B – each pair needs its own direct peering connection.

5.2 THE N^2 PROBLEM

Non-transitivity means connecting many VPCs together via peering alone requires a separate connection between *every pair* – 10 VPCs needing full interconnection would require 45 individual peering connections, and that number grows quadratically as more VPCs join. This quickly becomes unmanageable at any meaningful scale.

5.3 TRANSIT GATEWAYS: A CENTRAL HUB

A **Transit Gateway** solves this by acting as a central routing hub – every VPC connects once to the transit gateway, and the gateway handles routing between all of them, turning what would be N^2 peering connections into just N simple connections to one central point.

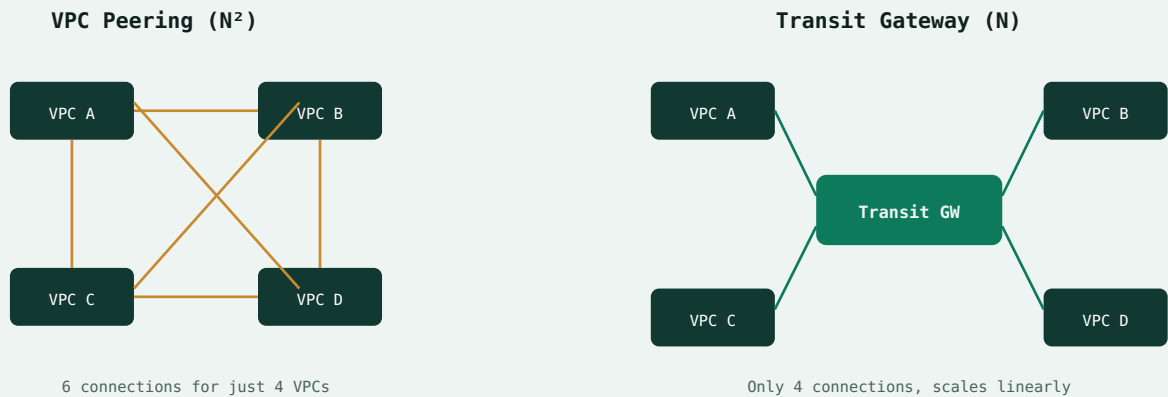


Fig 5.1 – VPC Peering needs a connection per pair (N^2 growth); a Transit Gateway needs just one connection per VPC (N growth).

Load Balancing & DNS in the Cloud

6.1 MANAGED LOAD BALANCERS

Cloud providers offer **managed load balancer services** implementing the same concepts covered in the Load Balancing guide – L4 and L7 distribution, health checks, TLS termination – but as fully managed infrastructure that automatically scales its own capacity, rather than something you provision, patch, and scale yourself. An **Application Load Balancer** (L7) routes based on HTTP content; a **Network Load Balancer** (L4) handles raw TCP/UDP at very high throughput with minimal latency.

6.2 CLOUD DNS & HEALTH-AWARE ROUTING

Managed DNS services extend basic domain resolution with **health-aware and latency-based routing** – automatically directing users to the closest healthy regional deployment, or failing over entirely to a backup region if an entire region becomes unavailable. This effectively adds a layer of load balancing and failover (Load Balancing guide, Sections 3–4) at the DNS level, above and before any individual load balancer even gets involved.

6.3 AUTO SCALING: THE CLOUD-NATIVE ADDITION

Perhaps the biggest practical difference from traditional on-prem load balancing: cloud load balancers commonly integrate directly with **auto scaling groups**, automatically adding or removing backend instances based on real-time load – the backend pool itself becomes elastic, growing and shrinking to match demand, something essentially impossible with physical on-prem hardware provisioned for peak capacity in advance.

Hybrid Connectivity: VPN & Direct Connect

7.1 SITE-TO-SITE VPN TO THE CLOUD

The simplest way to connect an on-premises network to a VPC is a **site-to-site VPN** – exactly the IPsec-based design covered in the VPN Protocols guide, Section 7, just with a cloud-managed VPN gateway on one end instead of a second physical office. Quick to set up, encrypted by default, but still travels over the shared public internet, inheriting its variable latency and bandwidth limitations.

7.2 DEDICATED PRIVATE CONNECTIONS

For higher bandwidth, lower and more consistent latency, and traffic that never touches the public internet at all, cloud providers offer dedicated physical connections – **AWS Direct Connect**, **Azure ExpressRoute**, **Google Cloud Interconnect** – a private, dedicated network link from your facility (or a colocation partner) directly into the cloud provider's network.

ASPECT	SITE-TO-SITE VPN	DEDICATED CONNECTION (DIRECT CONNECT / EXPRESSROUTE)
Setup time	Minutes to hours	Weeks to months – physical circuit provisioning
Path	Public internet, encrypted	Private, dedicated physical link
Bandwidth	Limited by internet connection and VPN overhead	Very high, consistent
Cost model	Low – often just data transfer charges	Higher – dedicated circuit + provider fees
Typical use	Smaller workloads, quick setup, DR/backup path	Large, steady, latency-sensitive enterprise workloads

Not mutually exclusive: many enterprises run a dedicated connection as the primary path for steady, high-volume traffic, with a site-to-site VPN configured as an automatic backup path if the dedicated circuit ever fails – the same redundancy principle as the active-passive failover design from the Load Balancing guide, Section 7, just applied to the connection itself rather than a server pool.

Worked Example: Building a Basic VPC

A simplified, Terraform-style walkthrough building the public/private subnet pattern from Fig 2.1 – a common baseline for a small web application.

8.1 CREATE THE VPC

```
$ resource "aws_vpc" "main" {  
$   cidr_block = "10.0.0.0/16"  
$ }
```

8.2 CREATE PUBLIC & PRIVATE SUBNETS

```
$ resource "aws_subnet" "public" {  
$   vpc_id = aws_vpc.main.id  
$   cidr_block = "10.0.1.0/24"  
$ }  
$ resource "aws_subnet" "private" {  
$   vpc_id = aws_vpc.main.id  
$   cidr_block = "10.0.2.0/24"  
$ }
```

8.3 ADD AN INTERNET GATEWAY & PUBLIC ROUTE

```
$ resource "aws_internet_gateway" "igw" {  
$   vpc_id = aws_vpc.main.id  
$ }  
$ resource "aws_route" "public_internet" {  
$   route_table_id = aws_route_table.public.id  
$   destination_cidr_block = "0.0.0.0/0"  
$   gateway_id = aws_internet_gateway.igw.id  
$ }
```

This route (Section 3.3) is what actually makes the public subnet "public" – everything else about the subnet's configuration is otherwise identical to the private one.

8.4 ATTACH A SECURITY GROUP

```
$ resource "aws_security_group" "web" {  
$   vpc_id = aws_vpc.main.id  
$   ingress {  
$     from_port = 443  
$     to_port = 443  
$     protocol = "tcp"  
$     cidr_blocks = ["0.0.0.0/0"]  
$   }  
$ }
```

A stateful security group (Section 4.1) allowing inbound HTTPS from anywhere – the reply traffic is automatically permitted back out without a separate outbound rule.

In practice: exact syntax varies by tool (Terraform, CloudFormation, Pulumi, or a provider's native CLI) and provider, but the underlying sequence – VPC, subnets, a gateway with a route, then security controls – is the near-universal pattern for standing up any basic cloud network.

Quick Reference & Glossary

TERM	DEFINITION
VPC / VNet	Virtual Private Cloud (AWS/GCP) / Virtual Network (Azure) – an isolated private network in the cloud.
CIDR Block	The overall private IP address range assigned to a VPC.
Subnet	A smaller carved-out address range within a VPC, often tied to one availability zone.
Route Table	The set of rules determining where a subnet's traffic is allowed to go.
Internet Gateway	Provides two-way (inbound + outbound) internet connectivity to a public subnet.
NAT Gateway	Provides outbound-only internet access for a private subnet.
Security Group	A stateful firewall attached to individual instances/interfaces.
NACL	Network ACL – a stateless firewall applied at the subnet level.
VPC Peering	A direct, non-transitive network connection between exactly two VPCs.
Transit Gateway	A central routing hub letting many VPCs interconnect without N² peering connections.
Direct Connect / ExpressRoute	A dedicated, private physical connection from on-premises into a cloud provider's network.
Auto Scaling	Automatically adding/removing backend instances based on real-time load.

End of guide. Cloud networking rewards the same fundamentals as physical networking – subnetting, routing, layered security – expressed through a provider's specific API and terminology. The concepts transfer completely between AWS, Azure, and GCP; only the exact names and defaults change.